Большакова Маргарита Вадимовна
Гр. 410902

Лабораторная работа № 2
P2P ЧАТ

Задание:

Необходимо разработать программу (консольную или графическую) для обмена текстовыми сообщениями, работающую в локальной сети в одноранговом режиме.

Каждый участник обмена сообщениями (узел) идентифицируется IP-адресом и произвольным именем, которое задается пользователем (через параметр командной строки, конфигурационный файл или любым другим способом). Уникальность имен не требуется.

Каждый узел с помощью UDP формирует список активных узлов (IP-адреса и имена):

после запуска узел отправляет широковещательный пакет, содержащий свое имя, для уведомления других узлов в сети о своем подключении к сети;

другие узлы, получившие такой пакет, устанавливают с отправителем TCP-соединение для обмена сообщениями и передают по нему свое имя для идентификации в чате.

В любой момент к чату может присоединиться новый клиент.

Обмен сообщениями ведется с помощью TCP в логически общем пространстве: каждый узел поддерживает по одному TCP соединению с каждым другим узлом и отправляет свои сообщения всем узлам в сети. Отключение узла должно корректно обрабатываться другими узлами.

Пользовательский интерфейс программы должен позволять вводить с клавиатуры и отправлять сообщения, а также просматривать историю событий с момента последнего запуска программы.

История должна включать следующие сообщения в хронологическом порядке с отметками времени:

- входящие сообщения от других узлов (с указанием имени и IP-адреса отправителя);
- собственные отправленные сообщения;
- обнаружение нового узла;
- отключение работающего узла.

Для обмена сообщениями рекомендуется разработать свой собственный формат сообщения, позволяющий передавать сообщения разных типов и упрощающих передачу сообщений в потоковом режиме, используемом в TCP.

Например:

смещение в байтах	тип сообщения
0	тип сообщения (1 - сообщение, 2 - передача имени, 3 - пользователь подключен, 4 - пользователь отключен т.п.)
1	длина сообщения в байтах далее n байт содержание сообщения

Способы тестирования чата в многопользовательском режиме:

Docker (для консольных программ).

Виртуальные машины VirtualBox (для графических программ).

Bind на разные адреса loopback интерфейса: 127.0.0.1, 127.0.0.2 и т. д.

Поддерживается в новых версиях Windows без дополнительных настроек, может потребовать дополнительных действий в Linux.

При создании сокетов на прослушивание входящих широковещательных UDP-пакетов и TCP-соединений нужно указывать IP-адрес, переданный при запуске программы (параметр командной строки, конфигурационный файл и т.п.). Далее запустить несколько экземпляров программы, передавая разные IP-адреса: 127.0.0.1, 127.0.0.2 и т.д. Все эти адреса будут ассоциированы с вашим компьютером, однако логически программы будут разными независимыми клиентами.

Ход работы:

1) Инициализация узлов и протокол обнаружения (UDP Broadcast)

Сначала программа просит ввести имя, IP-адрес и TCP-порт.

```
Введите ваше имя: марго
Введите ваш IP (настоящий адрес Wi-Fi): 192.168.0.124
Введите ваш TCP порт: 2509
```

```
Введите ваше имя: сава
Введите ваш IP (настоящий адрес Wi-Fi): 192.168.0.124
Введите ваш TCP порт: 2510
```

Создается UDP-сокет, для которого устанавливаются опции SO_BROADCAST (разрешение на отправку широковещательных пакетов) и SO_REUSEPORT (для локального тестирования нескольких узлов на одном ноутбуке).

Структура приветственного пакета:

- Байт 0: Тип сообщения (значение 3 означает «пользователь подключен»).
- Байт 1: Длина полезной нагрузки в байтах.
- Полезная нагрузка: Строка в формате Имя:TCP-порт.

Для надежности я добавила механизм «пульса»: программа отправляет такое приветствие каждые 3 секунды, чтобы новые пользователи нас заметили, даже если подключились чуть позже.

```
[22:28:34] ~~~~~~~ Привет, марго Твой IP: 192.168.0.124, TCP-порт: 2509 ~~~~~~~~
[22:28:34] UDP пакет для поиска друзей отправлен

Введите ваше сообщение:
```

```
[22:28:54] ~~~~~~~ Привет, сава Твой IP: 192.168.0.124, TCP-порт: 2510 ~~~~~~~~
[22:28:54] UDP пакет для поиска друзей отправлен

Введите ваше сообщение:
```

2) Прием UDP-пакетов и ведение списка узлов (запись в блокнот)

Параллельно в фоне работает функция listen_udp. Она постоянно слушает эфир. Если прилетает пакет, который начинается с цифры 3, программа понимает, что это кто-то новенький. Она достает из пакета имя и порт, проверяет, не свой ли это пакет, и если всё хорошо — записывает нового друга в словарь и выводит на экран надпись [+] К чату присоединился: ... с отметкой времени.

3) Отправка и прием текстовых сообщений (TCP)

Моя программа заранее открывает TCP-сервер на указанном порту и ждет звонков в режиме listen(). Когда кто-то пишет мне сообщение, сервер принимает соединение, читает текст, находит имя отправителя в словаре по его IP-адресу и выводит текст на экран.

Когда я пишу текст в консоль и нажимаю Enter, программа пробегается циклом по всем друзьям в словаре. Для каждого друга она создает новый клиентский TCP-сокет, стучится на его порт (connect()), отправляет текст и сразу же закрывает соединение (close()), чтобы не подвесить порты.

```
[22:28:54] [+] К чату присоединился: сава
Введите ваше сообщение:
[22:29:01] [сава] : привет
```

```
[22:29:11] [марго] : приветики
Введите ваше сообщение:
[22:29:14] [марго] : нормально
```

4) Анализ сетевого трафика в Wireshark

udp

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.0.124	192.168.0.255	UDP	43	2509 → 2509 Len=11
2	0.171598	192.168.0.124	192.168.0.255	UDP	41	2509 → 2509 Len=9
3	3.001296	192.168.0.124	192.168.0.255	UDP	43	2509 → 2509 Len=11
4	27.783536	192.168.0.124	192.168.0.255	UDP	49	2509 → 2509 Len=17
5	27.784083	192.168.0.124	192.168.0.255	UDP	49	2509 → 2509 Len=17
6	30.789062	192.168.0.124	192.168.0.255	UDP	49	2509 → 2509 Len=17

Тут видна реализация механизма «пульса»: пакеты летят ровно каждые 3 секунды (на колонке Time: ~~3, 27.78, 30.78~~) на правильный бродкаст-адрес 192.168.0.255.

IP-по 2509

```
> Frame 22: Packet, 56 bytes on wire (448 bits), 56 bytes captured (448 bits) on interface lo0,
> Null/Loopback
v Internet Protocol Version 4, Src: 192.168.0.124, Dst: 192.168.0.124
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
  > Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 52
    Identification: 0x0000 (0)
  > 010. .... = Flags: 0x2, Don't fragment
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 64
    Protocol: TCP (6)
    Header Checksum: 0x0000 [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 192.168.0.124
    Destination Address: 192.168.0.124
    [Stream index: 1]
> Transmission Control Protocol, Src Port: 49329, Dst Port: 2509, Seq: 1, Ack: 1, Len: 0

0000  02 00 00 00 45 00 00 34  00 00 40 00 40 06 00 00   ····E··4 ··@·@···
0010  c0 a8 00 7c c0 a8 00 7c  c0 b1 09 cd d4 2a fc 64   ···|···| ·····*·d
0020  95 60 ec da 80 10 18 ec  82 6f 00 00 01 01 08 0a   ·`····· ·o ·····
0030  f0 f1 4d ea ec 26 27 45                            ··M··&'E
```

tcp.port == 2509 or tcp.port == 2510

No.	Time	Source	Destination	Protocol	Length	Info
20	54.690671	192.168.0.124	192.168.0.124	TCP	68	49329 → 2509 [SYN] Seq=0 Win=65535 Len=0 MSS=16344 WS=64 TSval=4042345962 TSecr=0 SACK_PERM
21	54.690844	192.168.0.124	192.168.0.124	TCP	68	2509 → 49329 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=16344 WS=64 TSval=3961923397 TSecr=4042345962 SACK_
22	54.690867	192.168.0.124	192.168.0.124	TCP	56	49329 → 2509 [ACK] Seq=1 Ack=1 Win=408320 Len=0 TSval=4042345962 TSecr=3961923397
23	54.690887	192.168.0.124	192.168.0.124	TCP	56	[TCP Window Update] 2509 → 49329 [ACK] Seq=1 Ack=1 Win=408320 Len=0 TSval=3961923397 TSecr=4042345962
24	54.690893	192.168.0.124	192.168.0.124	TCP	68	49329 → 2509 [PSH, ACK] Seq=1 Ack=1 Win=408320 Len=12 TSval=4042345962 TSecr=3961923397
25	54.690899	192.168.0.124	192.168.0.124	TCP	56	2509 → 49329 [ACK] Seq=1 Ack=13 Win=408320 Len=0 TSval=3961923397 TSecr=4042345962
26	54.690929	192.168.0.124	192.168.0.124	TCP	56	49329 → 2509 [FIN, ACK] Seq=13 Ack=1 Win=408320 Len=0 TSval=4042345962 TSecr=3961923397

Строки 20, 21, 22 — это тройное рукопожатие (SYN, SYN, ACK, ACK).

Строка 24 — это отправка сообщения (PSH, ACK).

Строка 26 — это разрыв соединения (FIN, ACK),

```
> Frame 22: Packet, 56 bytes on wire (448 bits), 56 bytes captured (448 bits) on interface lo0,
> Null/Loopback
v Internet Protocol Version 4, Src: 192.168.0.124, Dst: 192.168.0.124
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
  > Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 52
    Identification: 0x0000 (0)
  > 010. .... = Flags: 0x2, Don't fragment
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 64
    Protocol: TCP (6)
    Header Checksum: 0x0000 [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 192.168.0.124
    Destination Address: 192.168.0.124
    [Stream index: 1]
> Transmission Control Protocol, Src Port: 49329, Dst Port: 2509, Seq: 1, Ack: 1, Len: 0

0000  02 00 00 00 45 00 00 34  00 00 40 00 40 06 00 00   ····E··4 ··@·@···
0010  c0 a8 00 7c c0 a8 00 7c  c0 b1 09 cd d4 2a fc 64   ···|···| ·····*·d
0020  95 60 ec da 80 10 18 ec  82 6f 00 00 01 01 08 0a   ·`····· ·o ·····
0030  f0 f1 4d ea ec 26 27 45                            ··M··&'E
```

- Это пакет ACK — третий и последний шаг установки соединения. Клиент подтверждает серверу, что готов передавать данные.
- Порт 2509 — это статический порт твоего сервера, который был задан в коде. А порт 49329 — это динамический порт. Операционная система (macOS) сама временно выдала этот случайный порт другу (отправителю) на время этого короткого соединения, чтобы знать, куда возвращать ответ.
- В поле Total Length (в разделе IPv4) написано 52 байта. Из них 20 байт — это заголовок IP-пакета (Header Length), и 32 байта — это заголовок TCP-пакета. Полезной нагрузки (текста) здесь 0 байт. Это доказывает, что пакет несет исключительно служебную функцию настройки соединения, а само сообщение полетит только в следующем пакете с флагом PSH.
- В разделе Internet Protocol Version 4 видно, что Src: 192.168.0.124 и Dst: 192.168.0.124 — абсолютно одинаковые. Это наглядно показывает, что тестирование проходило на одном ноутбуке, и операционная система замкнула маршрутизацию сама на себя, даже не выпуская пакет за пределы сетевой карты.

Вывод: в ходе выполнения лабораторной работы было успешно разработано многопоточное P2P-приложение, реализующее гибридную клиент-серверную архитектуру. Разработан собственный прикладной протокол: для автоматического обнаружения узлов применяется широковещательная рассылка через UDP-сокеты, а для гарантированной доставки текстовых сообщений устанавливается надежное соединение через TCP. Практическая реализация проекта позволила на глубоком уровне закрепить принципы многопоточного программирования, управления сетевыми гнездами (сокетами) и изучить особенности маршрутизации в локальных сетях с учетом масок подсетей и транспортных протоколов.